南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

Advanced Programming

Prof. Shiqi Yu (于仕琪)

yusq@sustech.edu.cn

http://faculty.sustech.edu.cn/yusq/

CC BY NC SA

Questions

In OpenCV there is a class `cv::Mat` for matrices. Find the answers to the questions from the source code of OpenCV at https://github.com/opencv/opencv

- How many data types does OpenCV's `cv::Mat` class support for matrix elements?
- How does `cv::Mat` handle data types when the size of `int` may not be 32 bits on different platforms?

BY NC SA

Hints

/modules/core/include/opencv2/core/hal/interface.h

- CV_8U -> CV_MAKETYPE

```
#define CV_CN_MAX     512
#define CV_CN_SHIFT   3
#define CV_DEPTH_MAX  (1 << CV_CN_SHIFT)
```

8

```
#define CV_8U   0
#define CV_8S   1
#define CV_16U  2
#define CV_16S  3
#define CV_32S  4
#define CV_32F  5
#define CV_64F  6
#define CV_16F  7
```

7

```
#define CV_MAT_DEPTH_MASK       (CV_DEPTH_MAX - 1)
#define CV_MAT_DEPTH(flags)     ((flags) & CV_MAT_DEPTH_MASK)
```

the least significant 3 bits

```
#define CV_MAKETYPE(depth,cn) (CV_MAT_DEPTH(depth) + (((cn)-1) << CV_CN_SHIFT))
#define CV_MAKE_TYPE CV_MAKETYPE

#define CV_8UC1 CV_MAKETYPE(CV_8U,1)
#define CV_8UC2 CV_MAKETYPE(CV_8U,2)
#define CV_8UC3 CV_MAKETYPE(CV_8U,3)
#define CV_8UC4 CV_MAKETYPE(CV_8U,4)
#define CV_8UC(n) CV_MAKETYPE(CV_8U,(n))

#define CV_8SC1 CV_MAKETYPE(CV_8S,1)
```

BY NC SA

Hints

/modules/core/src/matrix.cpp

00833f9 opencv / modules / core / src / matrix.cpp

Code Blame

```
657
658
659  void Mat::create(int d, const int* _sizes, int _type)
660  {
661      int i;
662      CV_Assert(0 <= d && d <= CV_MAX_DIM && _sizes);
663      _type = CV_MAT_TYPE(_type);
```

```
697          try
698          {
699              u = a->allocate(dims, size, _type, 0, step.p, ACCESS_RW /* ignored */, USAGE_DEFAULT);
700              CV_Assert(u != 0);
701              allocator = a;
702          }
```

CC BY NC SA

Hints

/modules/core/src/matrix.cpp

opencv / modules / core / src / **matrix.cpp**

Code Blame 1369 lines (1182 loc) · 36.3 KB

```cpp
class StdMatAllocator CV_FINAL : public MatAllocator
{
public:
    UMatData* allocate(int dims, const int* sizes, int type,
                       void* data0, size_t* step, AccessFlag /*flags*/, UMatUsag
    {
        size_t total = CV_ELEM_SIZE(type);
        for( int i = dims-1; i >= 0; i-- )
        {
            if( step )
            {
                if( data0 && step[i] != CV_AUTOSTEP )
                {
                    CV_Assert(total <= step[i]);
                    total = step[i];
                }
                else
                    step[i] = total;
            }
            total *= sizes[i];
        }
        uchar* data = data0 ? (uchar*)data0 : (uchar*)fastMalloc(total);
        UMatData* u = new UMatData(this);
```

/modules/core/include/opencv2/core/cvdef.h

opencv / modules / core / include / opencv2 / core / **cvdef.h**

Code Blame 950 lines (813 loc) · 35.6 KB

```cpp
\*****************************************************************************************/

#define CV_MAX_DIM              32
#define CV_MAT_CN_MASK          ((CV_CN_MAX - 1) << CV_CN_SHIFT)
#define CV_MAT_CN(flags)        ((((flags) & CV_MAT_CN_MASK) >> CV_CN_SHIFT) + 1)
#define CV_MAT_TYPE_MASK        (CV_DEPTH_MAX*CV_CN_MAX - 1)
#define CV_MAT_TYPE(flags)      ((flags) & CV_MAT_TYPE_MASK)
#define CV_MAT_CONT_FLAG_SHIFT  14
#define CV_MAT_CONT_FLAG        (1 << CV_MAT_CONT_FLAG_SHIFT)
#define CV_IS_MAT_CONT(flags)   ((flags) & CV_MAT_CONT_FLAG)
#define CV_IS_CONT_MAT          CV_IS_MAT_CONT
#define CV_SUBMAT_FLAG_SHIFT    15
#define CV_SUBMAT_FLAG          (1 << CV_SUBMAT_FLAG_SHIFT)
#define CV_IS_SUBMAT(flags)     ((flags) & CV_MAT_SUBMAT_FLAG)

/** Size of each channel item,
   0x28442211 = 0010 1000 0100 0100 0010 0010 0001 0001 ~ array of sizeof(arr_type_elem) */
#define CV_ELEM_SIZE1(type) ((0x28442211 >> CV_MAT_DEPTH(type)*4) & 15)

#define CV_ELEM_SIZE(type) (CV_MAT_CN(type)*CV_ELEM_SIZE1(type))
```

CC BY NC SA

Hints

modules/core/include/opencv2/core/mat.hpp

opencv / modules / core / include / opencv2 / core / **mat.hpp**

Code | Blame | 3863 lines (3302 loc) · 164 KB

```
2190
2191        enum { MAGIC_VAL  = 0x42FF0000, AUTO_STEP = 0, CONTINUOUS_FLAG = CV_MAT_CONT_FLAG, SUBMATRIX
2192        enum { MAGIC_MASK = 0xFFFF0000, TYPE_MASK = 0x00000FFF, DEPTH_MASK = 7 };
2193
2194        /*! includes several bit-fields:
2195             - the magic signature
2196             - continuity flag
2197             - depth
2198             - number of channels
2199         */
2200        int flags;
2201        //! the matrix dimensionality, >= 2
2202        int dims;
2203        //! the number of rows and columns or (-1, -1) when the matrix has more than 2 dimensions
2204        int rows, cols;
2205        //! pointer to the data
2206        uchar* data;
```

BY NC SA

`if` Statement

CC BY NC SA

if and if-else

- Statements are executed conditionally

```
int num = 10;
if (num < 5)
   cout << "The number is less than 5. " << endl;


if (num == 5 )
{
   cout << "The number is 5." << endl;
}
else
   cout << "The number is not 5." << endl;
```

BY NC SA

if-else if-else

```
if (num < 5)
    cout << "The number is less than 5." << endl;
else if (num > 10)
    cout << "The number is greater than 10." << endl;
else
    cout << "The number is in range [5, 10]." << endl;
```

CC BY NC SA

A little more complex

When will "Where I'm?" be printed?

How to make the code easier to understand?

```
if(num < 10)
if(num < 5)
cout << "The number is less than 5" << endl;
else
cout << "Where I'm?" << endl;
```

BY NC SA

? : operator

- When can we use the ternary conditional operator?

```
bool isPositive = true;
int factor = 0;
//some operations may change isPositive's value
if(isPositive)
   factor = 1;
else
   factor = -1;
```

```
factor = isPositive ? 1 : -1;
```

```
factor = (isPositive) * 2 - 1;
```

BY NC SA

Conditions

BY NC SA

Condition

- What should be a condition?

```
int num = 10;
if (num < 5)
    cout << "The number is less than 5. " << endl;
```

- The condition should be an expression which is convertible to bool
  - Its value can be `bool, char, int, float`

CC BY NC SA

Relational Expressions

- The condition can be a relational expression
- The 6 relational/comparison operators

Operator name	Example
equal to	`a == b`
not equal to	`a != b`
less than	`a < b`
greater than	`a > b`
less than or equal to	`a <= b`
greater than or equal to	`a >= b`

- Return 1 if the condition (such as a==b) is true,
- Return 0 if the condition is false.

CC BY NC SA

Logical Expressions

- If an operand is not `bool`, it will be converted to `bool` implicitly.

Operator name	Symbol-like operator	Keyword-like operator	Example
negation	`!`	`not`	`!a`
AND	`&&`	`and`	`a && b`
Inclusive OR	`||`	`or`	`a || b`

- Precedence: `!` > `&&` > `||`
- What's the value of the follow expressions?

```
if(-2 && true)
    cout << "The condition is true." << endl;

if(not -2)
    cout << " (!-2) is true, really?" << endl;
```

BY NC SA

Non-Boolean Expressions

- They will be converted to `bool` implicitly if it is feasible.

```
float count = 0.2f;
if (count) //not recommend to use a float-point number
    cout << "There are some." << endl;
```

- Pointers are also frequently used as conditions

```
int * p = new int[1024];
if (!p) // if(p == NULL)
    cout << "Memory allocation failed." << endl;
```

BY NC SA

`while` loop

CC BY NC SA

while loop

- Syntax :

```
while( expression )
{
    //...
}
```

- If the condition is true, the statement (loop body) will be executed.

while.cpp

```
int num = 10;
while(num > 0)
{
    cout << "num = " << num << endl;
    num--;
}
```

CC BY NC SA

do-while loop

- The test takes place after each iteration in a do-while loop.
- The test takes place before each iteration in a while loop.

while.cpp

```
int num = 10;
do
{
    cout << "num = " << num << endl;
    num--;
}while (num > 0);
```

CC BY NC SA

break statement

- Terminate a loop

while.cpp

```
int num = 10;
while (num > 0)
{
    if (num == 5)
        break;
    cout << "num = " << num << endl;
    num--;
}
```

BY NC SA

continue statement

- Skip the remaining part of the loop body and continue the next iteration.

while.cpp

```
int num = 10;
while (num > 0)
{
    if (num == 5)
        continue;
    cout << "num = " << num << endl;
    num--;
}
```

BY NC SA

The Condition, Be Careful!

- Can you find any problem from the code?

```
size_t num = 10;
while(num >= 0)
{
    cout << "num = " << num << endl;
    num--;
}
```

BY NC SA

The Condition, Be Careful!

```
bool flag = true;
int count = 0;
while(flag = true)
{
    cout << "count = " << count++ << endl;
    // and do sth
    if (count == 10) //meet a condition
    flag = false; //set flag to false to break the loop
}
```

CC BY NC SA

Why?

- Expression `3+4` has a value;
- Expression `a+b` has a value;
- Expression `(a==b)` has value (true or false);
- `a=b` is an assignment, also an expression and has a value

- The follow code can be compiled successfully!

```
int b = 0;
int m = (b = 8);
cout << "m="<< m << endl;
```

BY NC SA

`for` loop

CC BY NC SA

for loop

- Syntax

```
for (init-clause; cond-expression; iteration-expression)
    loop-statement
```

- Example

for.cpp

```
int sum = 0;
for(int i = 0; i < 10; i++)
{
    sum += i;
    cout << "Line " << i << endl;
}
cout << "sum = " << sum << endl;
```

CC BY NC SA

for loop VS while loop

```
int sum = 0;
for(int i = 0; i < 10; i++)
{
    sum += i;
    cout << "Line " << i << endl;
}
```

```
int sum = 0;
int i = 0;
while (i < 10)
{
    sum += i;
    cout << "Line " << i << endl;

    i++;
}
```

CC BY NC SA

for loop VS while loop

```
while(num > 0)
{
    cout << "num = " << num << endl;
     num--;
}
```

```
for(; num > 0; )
{
    cout << "num = " << num << endl;
    num--;
}
```

BY NC SA

Endless loop

- Sometimes we need it

```
for(;;)
{
    // some statements
    cout << "endless loop!" << endl;
}

while(true)
{
    // some statements
    cout << "endless loop!" << endl;
}
```

CC BY NC SA

`break/continue` statement

- break and continue statements behavior the same with while loops.

CC BY NC SA

`goto` and `switch` Statements

CC BY NC SA

goto Statement

- Jump to the desired location
- An unrecommended statement

goto.cpp

```
float mysquare(float value)
{
    float result = 0.0f;

    if(value >= 1.0f || value <= 0)
    {
        cerr << "The input is out of range." << endl;
        goto EXIT_ERROR;
    }
    result = value * value;
    return result;

  EXIT_ERROR:
    //do sth such as closing files here
    return 0.0f;
}
```

BY NC SA

`switch` Statement

- Execute one of several statements, depending on the value of an expression.
- `break` prevents executing some following statements. **Don't forget `break`!**
- More similar to `goto`, not `if-else if-else`

switch.cpp

```
switch (input_char)
{
    case 'a':
    case 'A':
        cout << "Move left." << endl;
        break;
    case 'd':
    case 'D':
        cout << "Move right." << endl;
        break;
    default:
        cout << "Undefined key." << endl;
        break;
}
```

BY NC SA

Questions

In OpenCV, the function `cv::add()` add two matrices. Find the source code to implement the addition at https://github.com/opencv/opencv

- How many data types does the function `cv::add()` support?
- For 8-bit unsigned int matrices, what will be the results when overflow happens?

BY NC SA